

문제를 발견하고,
흐름을 만들고,
운영을 지키는

엔지니어 김현호입니다.

01

문제를 이해하고

02

제품으로 구현하고

03

운영에서 검증하고

04

다시 개선합니다

Occount

학교 매점의 주문·결제를
완전 무인으로

키오스크 주문·결제부터 VAN 연동, 기기 운영과
장애 대응까지 직접 연결했습니다.

역할	프론트엔드 · 결제 흐름 구현
팀	5명
운영	약 6개월 실제 운영
기술	Spring Boot · MSA · Kafka · SSE

01

실제 화면이 운영을 증명합니다.

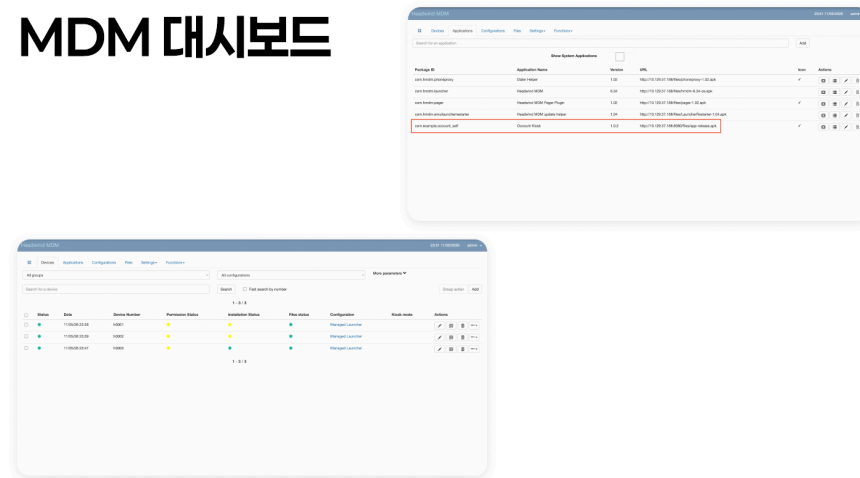
학생 인증부터 카드 결제, 키오스크 앱
배포와 기기 상태 확인까지 하나의
운영 흐름으로 구성했습니다.

키오스크 UI



키오스크 주문·결제 화면

MDM 대시보드



Headwind MDM 기반 기기 관리 화면

매점을 완전 무인으로 전환해 운영의 초점을 바꿨습니다.

기존 운영

학생



주문 확인



부원 개입



결제 처리

현재 운영

학생



키오스크



Smartro 단말



결제 완료

연속 스캔에서 발생한 중복 요청을 상태 기반 제어로 막았습니다.

문제	판단	구현	결과
연속 스캔 시 동일 상품 API 요청 반복	네트워크 로그에서 입력 이벤트 반복 확인	최근 요청 값과 진행 상태로 중복 입력 차단	불필요한 호출과 중복 상품 등록 방지

```
if (lastBarcode === barcode || requestInFlight) {  
  return;  
}
```

```
lastBarcode = barcode;  
requestInFlight = true;  
await requestProduct(barcode);
```

배운 점

외부 결제 서비스 연동에서는 기능뿐 아니라 예외 상태와 중복 입력을 함께 설계해야 제품의 안정성을 지킬 수 있습니다.

BUSAN MATHEMATICAL CULTURE CENTER · EXHIBITION

Media Art

부산수학문화관을 위한 인터랙티브 전시물

카메라가 본 몸을 숫자와 빛으로

부산수학문화관 전시물 제작 프로젝트로, 웹캠으로 포착한 사람의 실루엣을 AI와 픽셀 연산으로 해석해 관객이 직접 바뀌보는 경험으로 만들었습니다.

역할

기획 · 인터랙션 · 렌더링 구현

형태

Electron 기반 설치 작품

입력

웹캠 영상 · 사용자 조작

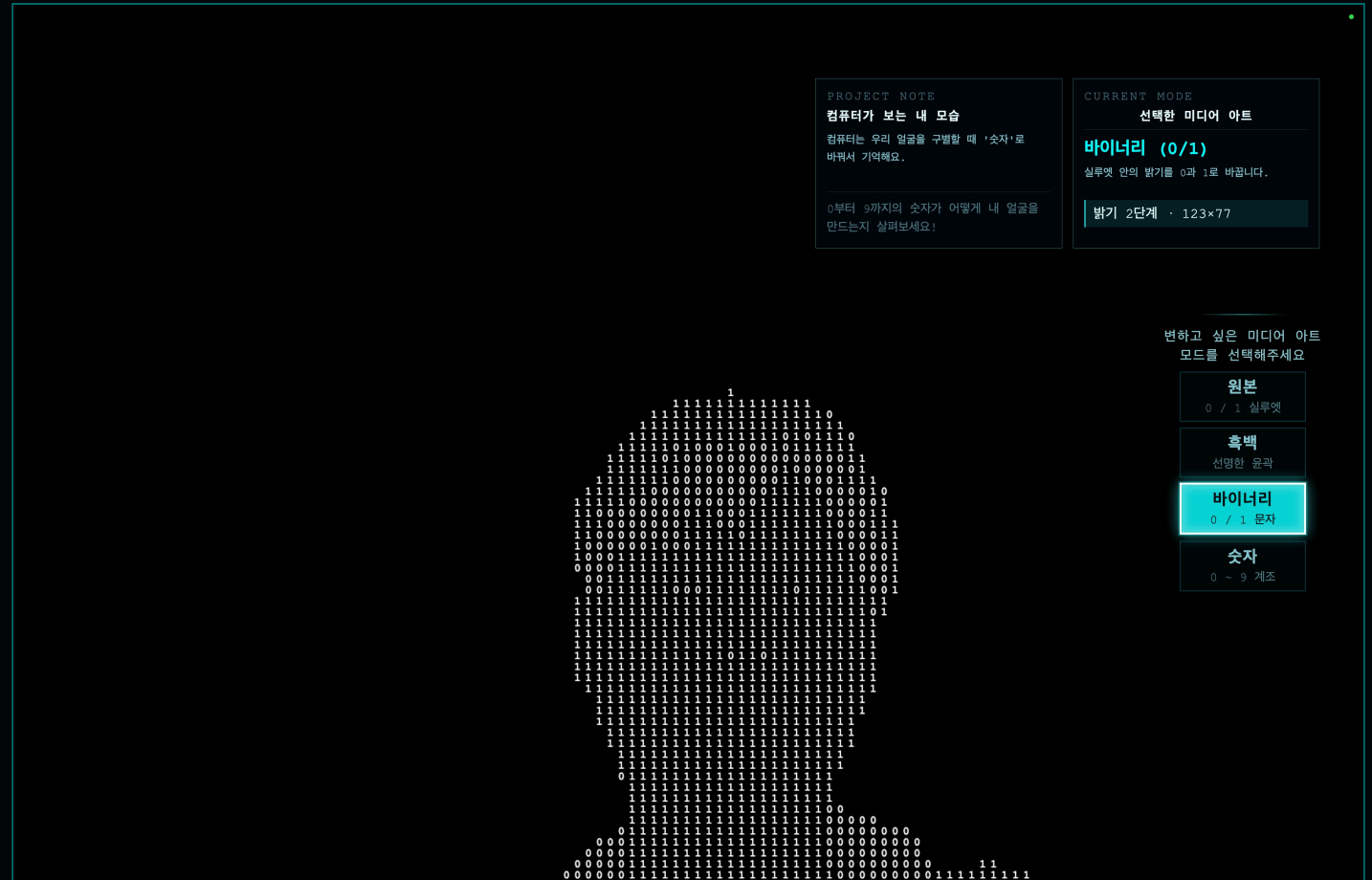
기술

JavaScript · Canvas · MediaPipe · GitHub Actions

프로젝트 저장소 보기 ↗

컴퓨터가 보는 내 모습

카메라가 인식한 사람 영역만 남기고,
픽셀의 밝기 데이터를 서로 다른
규칙으로 변환합니다. 관객은 같은
움직임이 숫자와 형태로 달라지는
과정을 바로 확인할 수 있습니다.

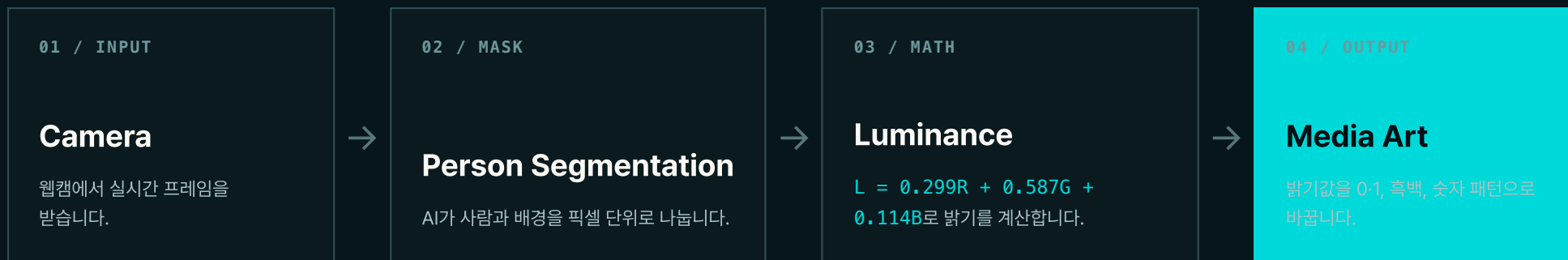


BINARY MODE

실시간 실루엣과 선택한 변환 모드를 한 화면에 보여주는 실제 작업 화면

움직임이 데이터가 되고, 데이터가 다시 장면이 됩니다.

복잡한 영상을 그대로 보여주는 대신, 인식·계산·표현의 단계를 분리해 관객이 이미지 처리의 원리를 읽을 수 있도록 구성했습니다.



03

같은 몸, 네 가지 해석

모드를 바꾸면 픽셀을 읽는 규칙이 바뀝니다. 결과를 선택하는 경험 자체가 이미지 처리 실험이 되도록 설계했습니다.

01

원본

CYBERPUNK SILHOUETTE

사이안 글로우와 0·1 표현으로 미래적인 분위기를 만듭니다.

02

흑백

MINIMAL SILHOUETTE

텍스트를 걷어내고 사람의 윤곽만 선명하게 남깁니다.

03

바이너리

0 / 1 MATRIX

밝기 기준으로 각 지점을 0 또는 1로 양자화합니다.

04

숫자

0 - 9 GRAYSCALE

밝기를 10단계로 나누어 숫자의 밀도로 표현합니다.

업데이트가 필요한 순간에도 전시를 멈추지 않도록.

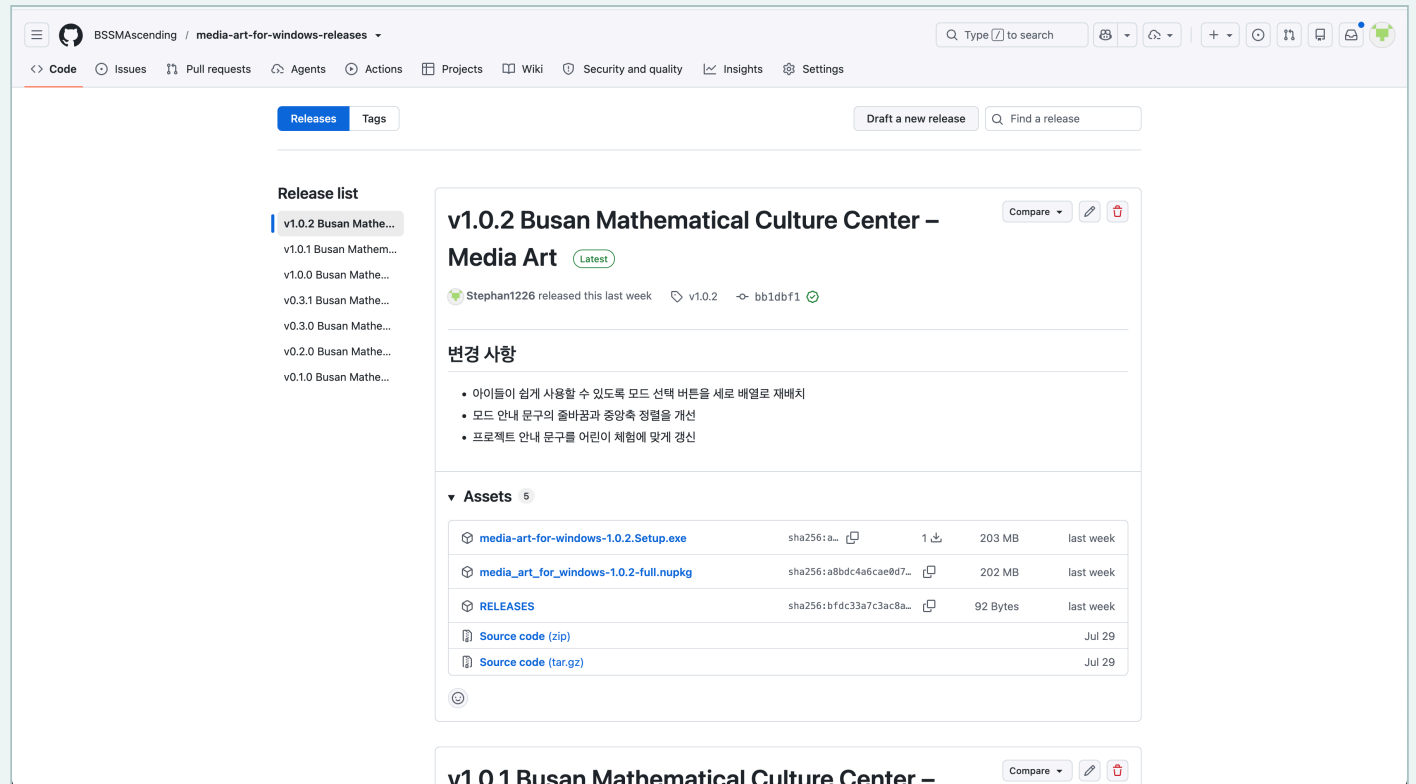
GitHub Release에 새 버전을 올리면
설치된 Windows 앱이 업데이트를
확인하고, 앱 안에서 다운로드 상태와
재시작 시점을 안내합니다.

감지 패키징된 Windows 앱이 실행
5초 후 새 릴리즈 확인

안내 다운로드 중 · 다운로드 완료
상태를 전시 화면에 표시

적용 '지금 재시작' 선택 시 업데이트
설치 후 앱 재실행

릴리즈 저장소 보기 ↗



GITHUB RELEASES

v1.0.2 Windows 설치 파일과 릴리즈 자산이 자동으로 게시된 실제 화면

전시장에서 계속 반응하도록 렌더링 구조를 다듬었습니다.

실시간성	조작 경험	전시 환경
렌더링과 AI 추론 주기를 분리	네이티브 버튼으로 모드 선택	카메라와 화면을 한 기기 안에서 처리
화면은 30fps를 목표로 갱신하고, 사람 인식은 12fps로 조절해 반응성과 계산량의 균형을 맞췄습니다.	키보드와 마우스로 접근할 수 있고, 선택된 상태를 시각·접근성 정보로 함께 노출했습니다.	외부 서버에 영상을 보내지 않고 로컬에서 인식과 Canvas 렌더링을 수행하도록 구성했습니다.

작품으로 배운 점

인터랙티브한 결과는 시각 효과만으로 완성되지 않습니다. 입력을 안정적으로 받고, 계산 단계를 설명하며, 관객의 선택이 지연 없이 결과에 닿도록 전체 흐름을 설계해야 합니다.

PERSONAL INFRASTRUCTURE

Home Server

서비스를 직접 배포하고
운영하는 나만의 인프라

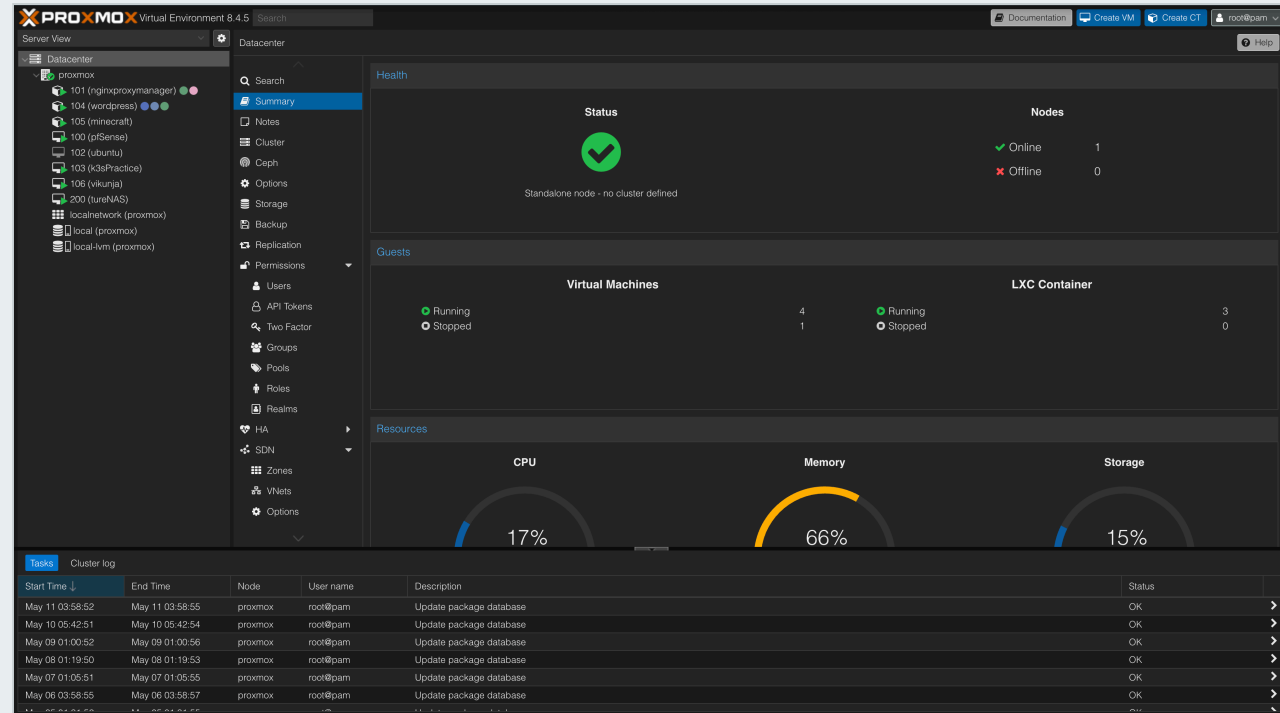
개발한 서비스가 실제로 동작하는 환경까지
이해하기 위해 가상화, 네트워크, 스토리지를 직접
설계하고 운영했습니다.

유형	개인 인프라 프로젝트
기간	2025.12 — 진행 중
구성	가상화 · 라우팅 · 스토리지
기술	Proxmox · Docker · Nginx Proxy Manager · TrueNAS

01

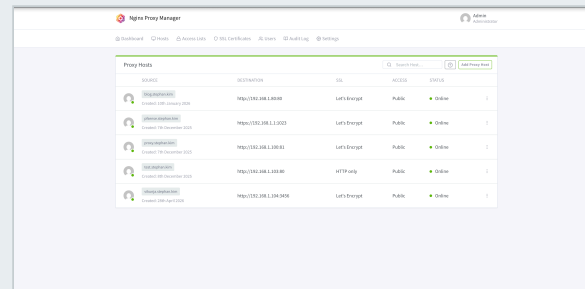
코드 밖의 운영 환경까지 직접 구축했습니다.

하나의 물리 서버 위에 서비스별 실행
환경을 분리하고, 외부 요청과 데이터
저장 흐름을 연결했습니다.



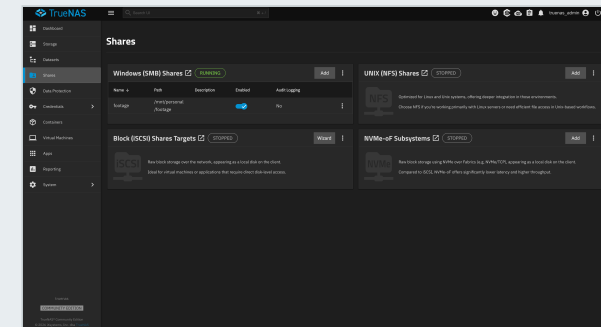
가상화

Proxmox에서 VM과 LXC 워크로드를 분리해 관리



라우팅

Nginx Proxy Manager로 도메인별 요청을 각 서비스에 전달

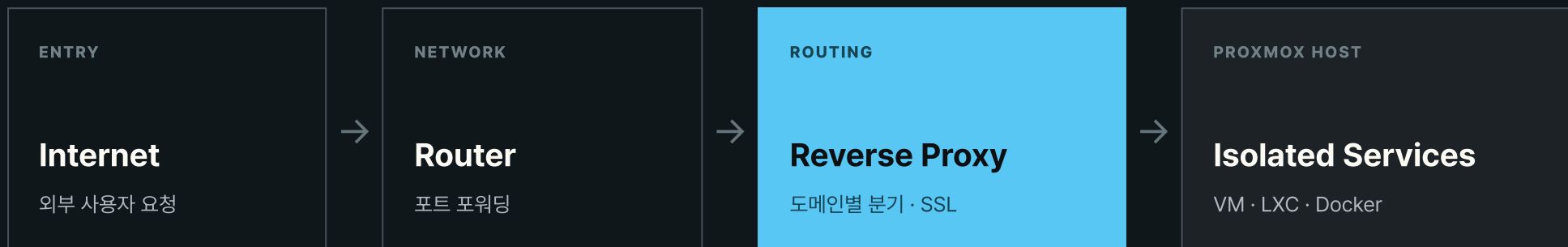


스토리지

TrueNAS 공유 스토리지로 데이터를 별도 관리

한 대의 서버를 여러 서비스의 운영 기반으로.

공인 IP로 들어온 요청을 Reverse Proxy에서 판별하고, Proxmox 내부의 독립된 실행 환경으로 전달하도록 구성했습니다.



01

서비스 간 영향 범위 축소

실행 환경을 분리해 배포와 재시작을 독립적으로 수행했습니다.

02

하나의 진입점으로 통합

여러 도메인의 요청을 한 서버에서 목적지에 맞게 분기했습니다.

03

운영 영역까지 확장

컴퓨팅, 네트워크, 저장소의 연결 구조를 직접 다뤘습니다.

접속 실패를 요청 경로를 따라 단계별로 추적했습니다.

증상	분석	조치	결과
내부에서는 열리지만 외부에서는 서비스 접속 불가	공인망부터 컨테이너까지 요청이 지나는 구간을 분리해 확인	포트 포워딩과 Reverse Proxy 목적지를 다시 구성	도메인으로 여러 서비스에 접근할 수 있는 운영 환경 완성

배운 점

서비스 운영에서는 애플리케이션 코드뿐 아니라 요청이 서버에 도달하고 데이터가 보존되는 전체 경로를 함께 이해해야 합니다.

김현호

배움은 빠르게, 실행은 끝까지.

동료와 현장에서 배운 것을 곧바로 실행하며, 더 오래 작동하는 제품을 함께 만들겠습니다.

GitHub 열기

이메일 보내기

DISCOVER.
BUILD.
OPERATE.

© 2026 김현호